

1.7 — Manage Session State, Resumption, and Forking

Concept

A **session** is a named, persistent conversation thread that can be resumed, branched, or summarized across time. Managing session state well is the difference between a stateless chatbot and a coherent long-running agent.

Named Sessions and Resumption

Claude Code identifies sessions by a **session ID**. The `--resume` flag reloads an existing session by ID, restoring the full message history and context:

```
claude --resume abc123
claude --session-id abc123
```

Programmatically, sessions are created and referenced via the Claude Agent SDK. Each session carries an ID you persist on your side — the SDK does not maintain a session registry for you.

Fork Session

`fork_session` creates a **branched copy** of an existing session at a specific point in time. The fork inherits all prior messages but diverges from that moment forward. The original session is unaffected.

Use forking when you need to:

- Explore a risky or speculative path without polluting the canonical session
- Run parallel sub-investigations from a common starting state
- Let a sub-agent work on a copy while the main agent continues on the original

Structured Summaries vs. Stale Context

Every context window has a finite token budget. Claude Code performs **context compaction** when a session's message history approaches the limit: it summarizes older turns into a compressed representation and discards the raw messages.

This introduces a critical distinction:

Structured Summary

Stale Raw Context

What it is An explicit, authored digest of key facts

Verbatim old messages that were never pruned

Risk Summary may omit edge-case details

Outdated decisions, superseded values, wrong state

Mitigation Author summaries at decision checkpoints

Prune or re-confirm state before resuming old sessions

When you resume a session that was last active days or weeks ago, the raw context may reference code, APIs, or decisions that have since changed. Always verify that the session's working assumptions still hold before allowing the agent to act.

Code Example

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

// --- Creating and persisting a named session ---
async function startNamedSession(userMessage: string): Promise<string> {
  const response = await client.beta.messages.create({
    model: "claude-opus-4-5",
    max_tokens: 8096,
    messages: [{ role: "user", content: userMessage }],
  });

  // Session ID returned in the response – persist this value
  const sessionId = response.id;
  await db.saveSession({ sessionId, createdAt: Date.now() });
  return sessionId;
}

// --- Resuming a session with a structured summary ---
async function resumeWithSummary(
  sessionId: string,
  summary: SessionSummary,
  newMessage: string
): Promise<Anthropic.Message> {
  // Inject a structured summary as the first user turn so the model
  // has accurate, compact context rather than relying on stale history
  const systemPrompt = buildSystemPrompt(summary);

  return client.beta.messages.create({
    model: "claude-opus-4-5",
    max_tokens: 8096,
    system: systemPrompt,
    messages: [
      {
        role: "user",
        content: `[RESUMING SESSION ${sessionId}]\n\n${newMessage}`,
      },
    ],
  });
}
```

```

// --- Forking a session for speculative exploration ---
async function forkForExploration(
  parentSessionId: string,
  explorationTask: string
): Promise<{ forkId: string; result: Anthropic.Message }> {
  // Record fork metadata so you can merge findings back or discard the fork
  const forkId = crypto.randomUUID();
  await db.saveSessionFork({ forkId, parentSessionId, createdAt: Date.now() });

  const parentMessages = await db.getSessionMessages(parentSessionId);

  const result = await client.beta.messages.create({
    model: "claude-opus-4-5",
    max_tokens: 8096,
    messages: [
      ...parentMessages,
      { role: "user", content: explorationTask },
    ],
  });

  return { forkId, result };
}

// --- Authoring a structured summary at a decision checkpoint ---
interface SessionSummary {
  goal: string;
  completedSteps: string[];
  pendingSteps: string[];
  keyDecisions: Record<string, string>;
  assumptions: string[];
  lastUpdated: number;
}

function buildSystemPrompt(summary: SessionSummary): string {
  return `
You are resuming an agentic session. The following structured summary captures all context you

GOAL: ${summary.goal}

COMPLETED: ${summary.completedSteps.join(", ")}
PENDING: ${summary.pendingSteps.join(", ")}

KEY DECISIONS:
${Object.entries(summary.keyDecisions)
  .map(([k, v]) => `  ${k}: ${v}`)
  .join("\n")}

```

```
ASSUMPTIONS (verify if resuming after >24h):
${summary.assumptions.map((a) => ` - ${a}`)}.join("\n")}
`.trim();
}
```

Common Mistakes

1. **Resuming old sessions without verifying assumptions** — Code, API contracts, and environment state may have changed since the session was created. A session from last week carries last week's worldview. Explicitly re-confirm critical assumptions before allowing the agent to act on them.
2. **Relying on compaction to preserve critical state** — Automatic compaction produces a generic summary optimized for token efficiency, not task fidelity. Author explicit structured summaries at decision checkpoints so that when the context is compacted, you control what survives.
3. **Using forks without tracking lineage** — Forked sessions that are never reconciled or discarded become orphaned context. Always record the `parentSessionId` with each fork so you can trace decisions back to their origin and clean up dead branches.
4. **Treating session ID as an opaque handle** — Session IDs are stable references but they carry no implicit guarantee that the model state is consistent with your application state. Pair every session ID with a metadata record that includes creation time, last-active time, and the current summary version.
5. **Injecting summaries as assistant turns** — Structured summaries belong in the `system` prompt or as the first `user` turn with a clear label. Injecting them as `assistant` turns confuses the model about what it has actually said and can cause self-referential errors.

References

- [Claude Code Session Management](#)
- [Context Window and Compaction](#)
- [Anthropic Agent SDK — Sessions](#)